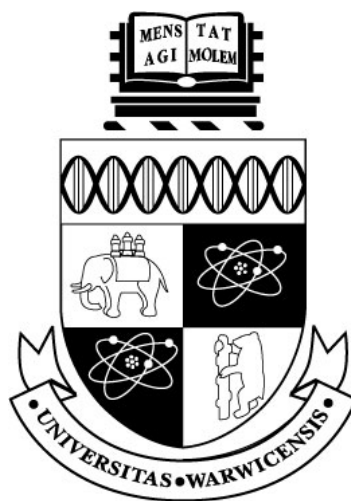




GODOT 4.6 大型可视化行为树的显示优化与实验评估

by

[作者姓名]

A thesis submitted in partial fulfilment
of the requirements for the degree of

游戏工程理学硕士

UNIVERSITY OF WARWICK

WMG

2026 年 9 月

目录

目录	ii
图目录	iv
表目录	v
致谢	vi
声明	vii
摘要	viii
1 引言	1
1.1 研究背景	1
1.2 问题定义与研究空白	1
1.3 研究目标与问题	2
1.4 研究目标与成功标准	2
2 文献综述	3
2.1 行为树	3
2.2 节点-连线树、层级和规模	3
2.3 局部避让与位置稳定	4
2.4 语义缩放与焦点上下文	5
2.5 关系强调与连线遮挡	5
3 研究方法	6
3.1 研究策略	6
3.2 实验行为树	6
3.3 屏幕尺寸配置	7
3.4 显示优化功能的实验方法	7
3.5 数据处理与判断标准	9
3.6 结构化开发者评价	9
3.7 插件功能验证	10

4	系统设计与实现	11
4.1	总体架构	11
4.2	资源模型与结构校验	11
4.3	执行系统与节点语义	12
4.4	编辑工作流与固定基础显示	12
4.5	Live Debug 与黑板检查	13
4.6	显示优化功能	13
4.6.1	智能拖拽重排	13
4.6.2	自适应缩放细节	14
4.6.3	可读连线覆盖	14
4.6.4	相关节点聚焦	14
4.6.5	鱼眼聚焦	14
4.7	测试游戏	15
5	实验结果	16
5.1	插件功能与可玩游戏验证	16
5.2	正式显示数据与安全检查	16
5.3	显示优化功能的启动前后比较	17
5.3.1	智能拖拽重排	17
5.3.2	自适应缩放细节	18
5.3.3	可读连线覆盖	18
5.3.4	相关节点聚焦	19
5.3.5	鱼眼聚焦	19
5.4	不同节点规模下的结果	20
5.5	不同屏幕尺寸下的结果	21
5.6	结构化开发者评价	22
6	讨论	23
6.1	平台正确性是显示研究的前提	23
6.2	显示优化作用的解释	23
6.2.1	不同复杂行为树规模下的优化效果	23
6.2.2	不同物理屏幕尺寸下的优化效果	24
6.2.3	哪些功能最有价值	24
6.2.4	证据能够说明什么	24
7	结论	26
	参考文献	27

图目录

4.1	插件、资源、执行系统、调试桥和测试游戏之间的关系	11
4.2	当前版本在 241 节点真实行为树上显示 Live Debug	13
4.3	五种规模行为树分别控制可玩场景中的五名敌人	15
5.1	智能拖拽重排关闭与开启：同一拖动位置下消除卡片遮挡	18
5.2	自适应缩放细节关闭与开启：相同概览缩放下减少卡片面积和字段 .	18
5.3	可读连线覆盖关闭与开启：相同路线透过卡片背景但避开文字	19
5.4	相关节点聚焦关闭与开启：选中节点及其关系分支保持突出	19
5.5	鱼眼聚焦关闭与开启：概览中恢复目标尺寸并淡化远处节点	20
5.6	Adaptive、Related Focus 和 Fisheye 随树规模变化的效果	20
5.7	Adaptive、Related Focus 和 Fisheye 随屏幕尺寸配置变化的效果	21

表目录

3.1	实验使用的行为树规模和主要游戏行为	6
3.2	三种屏幕尺寸及对应的编辑区域	7
3.3	显示优化功能的实验方法和比较内容	8
3.4	显示优化功能的主要指标与判断标准	9
4.1	当前运行节点的主要语义	12
5.1	当前版本的插件功能和游戏验证结果	16
5.2	显示优化功能开启前后的主要结果	17
5.3	五种行为树规模下的主要显示变化	20
5.4	三种物理尺寸配置下的主要显示变化	21
5.5	基于统一问题的结构化开发者评价	22

致谢

此处预留给作者感谢导师，以及帮助测试软件和审阅论文的人。提交前应由作者自行完成最终文字。

声明

本论文作为本人申请游戏工程理学硕士学位的材料提交给华威大学。论文由本人完成，且未曾用于申请其他学位。除明确引用的资料外，本文所述软件、测试数据和分析均来自本项目。本文没有报告任何参与者实验结果。正式提交前，作者必须根据学校规定审阅并个性化本声明，包括如实说明生成式人工智能的使用方式。

摘要

本项目制作了一个 Godot 4.6 可视化行为树插件，并把主要研究放在大型行为树的显示问题上。传统行为树编辑器在树变大后会占用很多画布，拖拽、连线 and 无关分支也容易同时干扰观察。为处理这些问题，当前项目制作了五项显示优化功能：智能拖拽重排、自适应缩放细节、可读连线覆盖、相关节点聚焦和鱼眼聚焦。

为了验证这些功能对于开发者在实际开发中造成的影响，项目通过五个真实的具有不同节点数量的行为树搭建了一个具有一定复杂度的游戏，并且使用了三种比较常见的不同尺寸的屏幕进行测试。通过不同的节点数量的行为树和不同尺寸的屏幕来进行测试，对比各个改进功能的实际优化效果。并且引入了实际的游戏开发者对于各个功能的评价。

结果显示，自适应缩放细节对不同树规模和屏幕尺寸都有较稳定的效果，因此最适合作为通用显示优化功能。智能拖拽重排能够处理节点移动产生的重叠，相关节点聚焦能够突出选中节点的结构范围。鱼眼聚焦在小屏幕和大型行为树中更加明显，但较强的局部放大也会增加部分节点重叠。可读连线覆盖能够改善被节点卡片遮挡的线段，但它主要适用于局部连线问题。

所有功能测试都满足预先设置的成功条件，并且没有改变行为树结构、保存坐标和同级节点的执行顺序。实验说明这些功能能够改善大型行为树的部分可测显示和编辑状态，也说明不同功能适合不同的开发环境。

关键词：行为树；Godot；节点-连线编辑器；语义缩放；焦点与上下文；图布局；游戏人工智能

1 引言

1.1 研究背景

本项目使用行为树控制游戏中的敌人。行为树把决策拆成复合节点、条件节点和动作节点，因此高层策略可以与移动、动画和战斗代码分开编辑。对于小型原型，全部逻辑写在一个脚本中仍然可行；但是状态和中断条件增加后，控制流会越来越难检查。行为树的层级结构可以缓解这个问题，也是它在游戏开发中的主要用途之一（Colledanchise and Ögren, 2018; Iovino et al., 2022）。

节点-连线树可以直接显示父子关系，但树越宽、越深，占用的画布也越大。行为树卡片还会显示参数、描述、Decorator、黑板键和运行状态，所以它通常比只显示名称的普通层级图更占空间。当画布里出现大量卡片时，开发者需要频繁缩放、平移和搜索，当前执行路径也容易被无关分支淹没。这个问题会直接影响节点定位、条件修改和失败检查，而不只是画面是否整齐。

因此，本项目先完成一个能够保存、运行和调试真实 NPC 决策的 Godot 插件，再用它测试当前保留的五项显示优化功能。这些显示优化功能分别处理拖动遮挡、缩放时的信息密度、卡片对连线的遮挡、选中节点的关系范围，以及概览中的局部细节。

1.2 问题定义与研究空白

已有研究从几个方向处理大型图。整洁树布局关注层级、顺序和紧凑性，而布局调整研究更关心消除重叠后能否保留原来的位置和拓扑（Reingold and Tilford, 1981; Sugiyama et al., 1981; Misue et al., 1995; Dwyer et al., 2006）。语义缩放会在缩小时减少显示内容，鱼眼视图则放大焦点并保留周围结构（Bederson et al., 1996; Furnas, 1986）。交互式强调和 EdgeLens 进一步处理相关子图与连线遮挡（Ware and Bobrow, 2005; Wong et al., 2003）。这些工作说明大型图可以从布局、信息密度、焦点和连线几个方面改进。

但是，这些研究没有直接说明游戏行为树编辑器应该怎样组合这些方法。行为树与普通目录不同，同级顺序可能决定执行优先级，叶节点会调用 Actor 代码，Decorator 也可能阻止分支运行。因此，显示优化功能不能只让画面看起来更整齐。它还必须保持执行顺序，支持撤销，并且不能破坏黑板和运行调试。

1.3 研究目标与问题

本文目标是在一个经过功能验证、能够驱动真实 NPC 的 Godot 4.6 可视化行为树平台上，设计并评估可组合的大型树显示优化。研究问题为：在不同的行为树节点规模和物理屏幕尺寸下，本项目提出的可组合显示优化，相较于传统基线显示，能否改善大型行为树的可测显示与编辑交互表现，同时保持行为树结构和执行语义不变？

本文的主要贡献包括三个方面。首先，本项目实现了一个完整的 Godot 4.6 可视化行为树平台，使真实游戏行为能够在同一系统中完成编辑、运行和调试。其次，项目针对大型行为树的显示问题设计并实现了五项可以独立比较的显示优化功能。最后，项目使用五棵真实游戏行为树和三种物理屏幕尺寸进行实验，比较不同显示优化功能在不同环境中的作用与代价，并据此说明哪些显示优化功能适合作为通用功能，哪些只适合特定使用场景。

1.4 研究目标与成功标准

项目工作分为平台建设与研究评估。平台建设包括：实现可序列化的树、节点、Decorator 和类型化黑板资源；实现 Root、Sequence、Selector、Reactive Selector、Random Selector、Parallel、Repeat、Wait、Action 和 Condition 的 success、failure 与 running 语义；允许可复用组件为 NPC 指定树和 Actor，并由叶节点调用 Actor 方法；提供创建、连接、断开、拖拽、框选、多节点移动、类型化编辑、校验、保存、加载、撤销和重做。

在进行显示优化实验前，首先需要确认行为树插件的基础功能正确。各类节点能够按照预期执行，编辑后的行为树能够正确保存和重新打开，撤销与重做不会破坏树的内容；五个可玩敌人应分别加载 31、61、121、241 和 364 节点树，玩家拥有无限生命，敌人数量有限，击败全部敌人后游戏能够结束。

显示研究的成功标准分为三类。第一，功能必须产生它声称的直接变化，例如 Smart Drag 减少拖动造成的重叠，Adaptive 降低概览卡片面积，Related Focus 淡化无关节点。第二，功能不能改变树拓扑、Decorator 归属或同级执行顺序。第三，关闭功能后不能留下尺寸、透明度、边框或临时布局残留。如果测试过程中出现程序报错或意外停止，即使其他检查通过，这次测试仍然算作失败。

2.1 行为树

较早的行为树研究主要围绕游戏人工智能展开。Flórez-Puga 等（2009）在游戏行为树中加入查询和已有行为复用，Lim 等（2010）则使用行为树为商业即时战略游戏 DEFCON 制作智能体。这些研究说明，行为树在早期就已经被用于组织包含多个条件和动作的游戏决策，而不只适合简单演示。

此后，相关研究开始进一步说明行为树的执行方式和理论特点。Marzinotto 等（2014）提出了用于机器人控制的统一行为树框架，Colledanchise 与 Ögren（2017, 2018）又从混合控制和模块化角度分析了行为树。到 Iovino 等（2022）进行综述时，行为树研究已经覆盖游戏人工智能、机器人、自动学习、规划和设计工具等多个方向。这一发展过程说明，行为树已经从游戏中的实用工具逐渐发展为一种更通用的智能体控制方法。

在测试显示优化功能之前，必须先确认行为树本身能够正确运行，否则无法判断测试中出现的问题来自显示优化功能还是行为树运行系统。在本项目的实现中，Sequence 可以记住当前执行位置，而 Reactive Selector 需要重新检查高优先级条件；Random Selector 应在一次运行期间保持同一个选择；Repeat 与 Wait 也必须在重启时清除记忆。黑板用于在感知、条件和动作之间共享状态，Decorator 用于在不修改子节点本身的情况下限制或改变分支执行。编辑器不能把节点顺序和 Decorator 当成纯视觉信息，因为它们会影响执行结果。

2.2 节点-连线树、层级和规模

节点 - 连线图适合显示行为树，因为连线可以直接表示父子关系。Reingold 与 Tilford（1981）提出的整洁树布局强调父节点居中、同层间距和子树分离，Sugiyama 等（1981）进一步讨论同层排序和节点位置。这些方法很适合自动生成一棵整齐的树，但没有完全解决交互式编辑的问题，因为用户仍然希望自由拖动节点。

布局是否更好不能只凭画面是否整齐判断。Purchase（1997）发现边交叉在所测试的图形美学中影响较大；Ware 等（2002）又说明路径连续性、交叉和分支会影响节点-连线图中的路径查找。Purchase、Carrington 与 Allder（2002）的结果进一步表明，只有部分美学指标会稳定影响任务表现，而且图的语义领

域也会改变结果。

规模增加后，节点和连线会同时变得更难观察。Ghoniem 等（2005）比较了节点-连线图与矩阵表示，并说明结果会同时受到规模、密度和任务影响。SpaceTree 把动态布局 and 选择性显示加入节点-连线树，也说明表示方法和交互方式需要一起评价（Plaisant et al., 2002）。对于行为树而言，节点-连线方式仍然最容易表达父子方向和同级顺序。

节点式界面也被用于组织其他复杂的开发流程。InstructPipe 允许用户生成机器学习流程，并继续在节点图中进行修改（Zhou et al., 2025）。它研究的对象虽然不是行为树，但同样需要处理节点位置、连接关系和复杂流程导航，说明这些问题并不只存在于行为树编辑器中。

布局设计还需要考虑不同领域自身的结构要求。Helmke 等（2024）发现，领域专家有时会优先保留图中的技术结构，而不是一律减少连线交叉或边长。行为树同样具有自己的结构要求，例如父子方向和同级执行顺序，因此自动布局不能只追求占用面积更小。Di Bartolomeo 等（2024）对图布局评价的系统综述还指出，布局算法的评价需要同时说明数据、规模和使用的指标。这为本文使用真实行为树、多个节点规模和多项直接指标提供了方法依据。

2.3 局部避让与位置稳定

交互式编辑与一次性自动布局不同。用户拖动节点后，如果系统重新排列整棵树，重叠可能会消失，但原来的位置记忆也会被破坏。Misue 等（1995）把它概括为布局调整和心理地图问题。Dwyer 等（2006）提出，节点需要彼此分离，同时新位置应尽量接近原位置；后续研究又说明，在加入约束时仍然需要保留原来的拓扑（Dwyer et al., 2009）。

行为树卡片有真实宽高、文字和标签，所以它们不能只被当作没有面积的点。Gansner 与 Hu（2010）强调，消除重叠时应尽量保留邻近关系和原图形状。这项研究给 Smart Drag 提供了方向，系统应该处理新重叠，但不应该借此接管整张画布。

本项目没有直接复制某一种完整算法，而是限制启动距离、刷新频率、传播层数和最大影响范围，再使用父子和同级关系决定哪些节点一起避让。系统只保持拖动前已经成立的父上子下关系。

Archambault 与 Purchase（2013）的实验表明，保持节点位置是否有帮助会受到具体任务的影响。因此，本项目要求重新布局时尽量维持节点之间的相对关系，并减少不必要的位置变化，但不会仅凭这一点认定功能改善了编辑体验。

2.4 语义缩放与焦点上下文

语义缩放和鱼眼都用于处理有限画布中的大量信息，但采用了不同的方法。语义缩放会根据缩放级别改变显示内容，而不只是把所有内容按比例缩小。Pad++ 展示了界面内容可以随缩放级别发生变化（Bederson et al., 1996），Summers 等（2003）也在程序可视化中比较了不同的语义缩放方式。缩小时隐藏次要字段、放大后恢复详细信息，是这类方法的主要特点。

鱼眼属于焦点与上下文方法。它会放大用户当前关注的区域，同时保留周围结构。Furnas（1986）提出使用兴趣度区分焦点和周围内容，Sarkar 与 Brown（1994）将相近思想用于图形鱼眼。Wang 等（2019）进一步研究了大型图中鱼眼放大可能造成的结构失真，并提出了结构感知的方法。

这些显示方法并不一定适合所有任务。Cockburn 等（2009）的综述指出，概览、缩放和焦点与上下文方法各有优点和代价。Büring 等（2006）在小屏幕上的实验也没有发现鱼眼一定能够缩短任务时间，虽然更多参与者表示更喜欢这种显示方式。此外，Jakobsen 与 Hornbæk（2013）说明，物理屏幕尺寸、信息空间和显示比例会共同影响交互结果。

这些研究为语义缩放、鱼眼和屏幕尺寸提供了设计依据，但不能直接说明它们在大型行为树编辑器中的效果。因此，本文将语义缩放和鱼眼作为两项不同功能，并在不同树规模和屏幕尺寸下分别进行比较。

2.5 关系强调与连线遮挡

大型图不一定要先隐藏节点。Ware 与 Bobrow（2005）使用交互式强调帮助用户查询节点-连线图，van Ham 与 Perer（2009）则从兴趣目标出发保留上下文。对于行为树，这类方法的好处是可以保留原来的空间位置，同时降低无关分支的视觉强度。本项目的 Related Node Focus 因此使用边框和透明度区分选中节点、祖先、后代、同级节点和无关分支。

连线也可能被节点或其他边挡住。Wong 等（2003）的 EdgeLens 会弯曲焦点附近的连线，为拥挤区域留出空间；Holten（2006）的层级边捆绑则沿层级组织大量连线。McGee 与 Dingliana（2012）的实验说明，捆绑可能帮助观察总体聚类，却可能降低单条路径追踪的速度和准确率。减少视觉杂乱因此不等于所有连线任务都会更容易。

前面的研究说明，布局调整、语义缩放、鱼眼、高亮和连线处理都可以缓解大型图的部分显示问题。但是，这些研究使用的图类型、任务和评价条件并不相同，因此不能直接说明这些方法在大型行为树编辑器中的实际效果。行为树还需要保持父子结构、同级执行顺序和节点信息，这使它与普通节点图存在一定区别。基于这一研究空白，本文将在不同树规模和屏幕尺寸下分别比较这些显示优化功能，检查它们产生的直接变化、适用环境和主要代价。

3 研究方法

3.1 研究策略

本研究先确认 Godot 4.6 行为树插件能够正常编辑和运行，再测试用于大型行为树的显示优化功能。研究重点是这些显示优化功能在不同情况下能够产生多大的效果，以及它们更适合哪些情况。

测试分为两部分，第一部分比较单项或组合使用的显示优化功能在不同情况下的实际效果。在相同的行为树、屏幕尺寸和操作目标下，程序分别记录功能开启前后的结果，从而判断各项显示优化功能是否达到预期作用，以及它们在不同节点规模和屏幕尺寸下的效果差异。第二部分由项目开发者实际使用这些显示优化功能，并按照同一组问题记录每项功能有什么帮助、存在什么问题，以及适合在什么情况下使用。由于这部分只有项目开发者本人参与，因此只能作为使用感受的补充，不能代表其他游戏开发者的意见。

3.2 实验行为树

实验使用当前可玩游戏中的五棵真实行为树，节点数量分别为 31、61、121、241 和 364。它们代表从小型到大型的不同规模和复杂程度。

表 3.1: 实验使用的行为树规模和主要游戏行为

节点数量	树中包含的主要游戏行为
31	躲避、方向近战、直接追击、返回守卫位置、巡逻和待机
61	低血量撤退与治疗、躲避、方向近战、跳跃越障、中距离移动、追击、搜索玩家最后位置、返回和巡逻
121	撤退与治疗、防御、近战、跳跃越障、梯子攀爬、远程攻击、中距离移动、追击、搜索、返回、巡逻和待机
241	包含 121 节点树的主要行为，并提供更多恢复、防御、近战、远程攻击、追击、搜索、巡逻和待机方案
364	包含前述主要行为，并增加防守反击、连续近战、组合越障、移动射击和多路线追击等复杂战术组合

这些行为树从较简单的决策逐渐增加到包含大量分支的复杂决策，可以用来观察显示优化功能是否会随着树的扩大而产生不同效果。实验会在每棵树上使用相同的方法，分别比较显示优化功能开启前后的画面变化。五棵树不仅大小不同，内部结构也不同。因此，实验可以比较它们的实际显示效果，但不能把所有结果差异都简单地归结为节点数量增加。不过，它们能够反映插件在真实游戏行为树从小型逐渐扩大到大型时的实际表现。

3.3 屏幕尺寸配置

实验设置了三种屏幕尺寸，分别约为16英寸、27英寸和32英寸。插件视窗除了行为树画布，还包含工具栏、菜单和其他界面内容，因此实际能够显示和编辑节点的区域会更小。实验在判断节点是否位于画面内时，使用的是实际编辑区域。

表 3.2: 三种屏幕尺寸及对应的编辑区域

屏幕条件	物理尺寸	屏幕大小	插件视窗大小	实际编辑区域
小型屏幕	34 × 22 cm	15.94 英寸	1190 × 770	858 × 642
中型屏幕	60 × 33 cm	26.96 英寸	2100 × 1155	1768 × 1027
大型屏幕	70 × 39 cm	31.55 英寸	2450 × 1365	2118 × 1237

不同尺寸的屏幕能够显示的节点数量不同。在小型屏幕上，大型行为树需要缩得更小才能看到整体；在较大的屏幕上，可以同时显示更多节点，但大型行为树仍然可能超出编辑区域。

实验在三种屏幕条件下使用相同的行为树和测试操作，以比较屏幕大小是否会影响显示优化功能的作用。

3.4 显示优化功能的实验方法

实验分别测试各项显示优化功能。每次测试都使用同一棵行为树、同一种屏幕条件和同一个测试对象，先记录功能关闭时的结果，再记录功能开启后的结果。这样可以直接看出该功能带来的变化。

每次测试开始前，程序都会把画布恢复到相同的初始状态，避免上一次测试造成的节点移动、透明度或缩放变化影响下一次结果。功能开启后，程序会等待画面完成更新，再记录数据。

一般情况下，程序先按照节点在画布上的横向位置从左到右排列，然后从约22%、50%和78%的位置选择三个节点。这三个节点大致位于行为树的左侧、

中间和右侧。根节点以及无法产生有效聚焦效果的节点不会被选择。相同的目标节点会用于所有屏幕条件以及功能开启前后的比较。

表 3.3: 显示优化功能的实验方法和比较内容

功能	实验方法	比较内容
智能拖拽重排	优先选择同一个父节点下相邻的两个子节点，其中一个是没有子节点的叶节点。程序将叶节点拖到另一个节点的位置附近，主动形成节点遮挡。关闭功能时保留遮挡，开启功能时允许周围节点自动避让。两次测试使用完全相同的拖动路径。	比较遮挡是否被消除、多少个周围节点发生移动、这些节点移动了多远，以及距离较远的分支是否受到影响。
自适应缩放细节	在相同的概览缩放比例下，把固定目标节点移动到画面中央。关闭功能时，所有节点仍然显示完整卡片；开启功能后，节点会根据缩放比例减少次要文字并缩小卡片。	比较所有节点占用的总面积、显示的文字字段数量、能够完整显示在编辑区域中的节点数量，以及是否出现父节点跑到子节点下方的问题。
可读连线覆盖	从行为树中选择三条容易被其他节点挡住的父子连线。如果原有布局中没有合适的情况，程序会临时移动一张不属于连线两端的节点卡片，形成相同的遮挡场景。关闭功能时卡片背景完全不透明；开启后背景不透明度降为 72%，但文字仍保持不透明。	比较被节点挡住的连线是否能够显示出来，同时检查节点文字是否仍然清楚，以及连线原来的经过路线是否发生改变。
相关节点聚焦	前两个测试案例分别选择一个固定节点，第三个案例同时选择两个固定节点。关闭功能时只保留普通的选中状态；开启功能后，选中节点使用白色边框，父节点和子节点使用黄色边框，同级节点使用绿色边框，其他节点和连线变淡。	比较无关节点是否正确变淡、相关节点是否使用正确的边框、画面中保持醒目的节点数量是否减少，以及节点位置是否保持不变。
鱼眼聚焦	鱼眼主要用于缩小后的行为树，因此关闭和开启鱼眼时都保留自适应缩放。程序把鼠标位置固定在目标节点上，关闭状态不放大节点，开启状态则放大目标及其附近节点。	比较目标节点放大了多少、恢复了多少文字信息、远处节点是否缩小和变淡，以及放大后是否产生新的节点遮挡。

前四项功能在关闭状态下不会开启其他实验功能。鱼眼聚焦是唯一的例外，因为它需要在缩小后的概览画面中使用。因此，鱼眼实验比较的是“只有自适应缩放”和“自适应缩放加鱼眼聚焦”之间的差异。

每项显示优化功能都会在 5 棵行为树、3 种屏幕尺寸和 3 组固定测试案例中进行比较，因此每项显示优化功能包含 45 组开启前后对比。全部显示优化功能一共形成 225 组对比。由于每组对比都包含一条关闭记录和一条开启记录，所以最终共有 450 条实验记录。

3.5 数据处理与判断标准

数据处理时，将屏幕尺寸、行为树规模和测试案例相同的关闭记录与开启记录配对，计算功能开启前后的差值和变化比例。结果分别按照显示优化功能、行为树规模和屏幕尺寸进行整理，并报告平均值、中位数、变化范围和达到预设标准的案例数量。

表 3.4: 显示优化功能的主要指标与判断标准

功能	主要指标	实验成功判断标准
智能拖拽重排	重叠面积、自动移动节点数量和移动范围	至少消除 95% 的重叠面积，自动移动的其他节点不超过 24 个
自适应缩放细节	节点总面积、显示字段数量和父子节点位置	节点面积或字段数量减少，并且不能增加父子节点高低关系错误
可读连线覆盖	被遮挡连线的显示情况、文字显示和连线路线	被遮挡的连线能够部分显示，节点文字保持清晰，原有连线路线不变
相关节点聚焦	无关节点淡化比例和相关节点显示状态	所有无关节点正确变淡，选中节点及相关节点保持清晰
鱼眼聚焦	目标节点宽度、显示字段和局部调整范围	目标节点至少增大 5% 并显示更多字段，自动调整不超过 8 个附近节点

所有实验还需要保证行为树本身不受显示优化功能影响，包括节点数量、连接关系、执行顺序和保存位置。关闭功能后，节点大小、透明度、边框和临时位置变化也必须恢复，实验运行过程中不能出现错误或异常停止。

鱼眼聚焦主要用于查看局部内容，因此不要求整个画布始终没有重叠。它造成的新重叠会被记录下来，并在结果和讨论中单独说明。

3.6 结构化开发者评价

自动实验完成后，本项目开发者会直接在 Godot 编辑器中使用这些显示优化功能。由于评价者同时也是使用行为树制作游戏的开发者，因此评价会从实际查看和编辑行为树的角度进行。

评价会覆盖五棵行为树和三种屏幕尺寸。在每种情况下，开发者会完成与该功能有关的查看或编辑操作，并记录实际使用后的反馈。

评价内容包括：

- 该功能主要帮助完成什么操作；
- 开启后是否更容易查看或编辑行为树；
- 使用过程中出现了什么问题；
- 该功能更适合哪种行为树规模；
- 该功能更适合哪种屏幕尺寸；

这些反馈会与自动实验的数据一起使用。自动实验说明画面发生了哪些变化，开发者评价则说明这些变化在实际使用中是否有帮助，以及可能带来哪些使用问题。最终根据两部分结果，判断每项功能更适合怎样的使用环境。

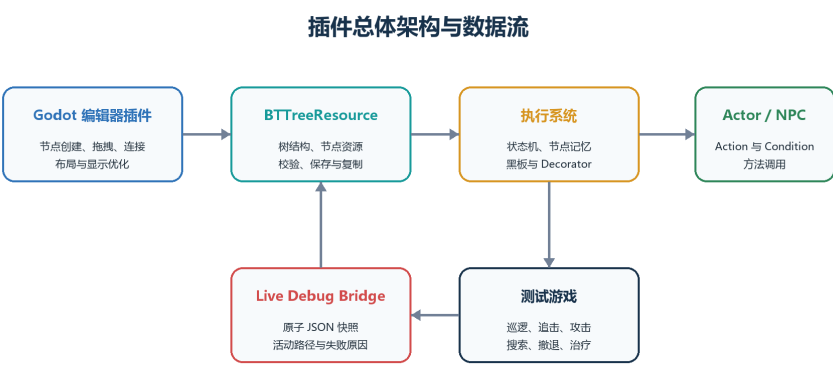
3.7 插件功能验证

在进行显示实验前，需要先对行为树插件的基本功能进行必要验证。验证内容包括节点是否能够正确执行，节点的创建、连接和编辑是否正常，行为树能否正确保存和重新加载，以及行为树能否控制游戏中的敌人完成预定行为。这部分验证只用于确认实验平台能够正常工作。

系统设计4与实现

4.1 总体架构

系统由编辑器插件、资源模型、执行系统、调试桥和测试游戏五部分组成。编辑器插件运行在 Godot 编辑器内，负责把行为树资源显示成可操作的 GraphEdit 画布。资源模型保存节点、父子关系、Decorator、黑板 Schema 和节点坐标。执行系统读取同一资源并把 Tick 结果传递给游戏 Actor。调试桥把当前状态、失败原因和黑板值送回编辑器。



编辑器负责可视化建模，执行系统负责节点判断与动作调用，Live Debug 将执行状态回传到编辑器。

图 4.1: 插件、资源、执行系统、调试桥和测试游戏之间的关系

这个划分使同一棵树可以同时用于编辑、保存、自动测试和游戏运行。显示优化功能只修改卡片的临时尺寸、透明度、边框或画布偏移，不改写节点类型和连接。用户真正拖动并释放的节点新坐标会正常保存，并可以撤销或重做；为了避让而自动移动的其他卡片不写回资源。

4.2 资源模型与结构校验

行为树资源保存根节点和节点集合。每个节点有稳定身份、类型、编辑器坐标、参数和子节点引用。Decorator 附着在拥有者节点上，而不是作为独立画布卡片。黑板 Schema 定义键名、类型和默认值，Action 与 Condition 通过类型化控件选择方法、键、比较运算和值。

保存前的校验会拒绝多个根、环、断开的非法引用、错误的子节点数量、无效 Decorator 归属和不符合 Schema 的黑板键。加载后再次建立节点卡片和连接。同级节点按照横坐标从左到右排序，这个顺序会影响 Sequence、Selector 和其他有序复合节点的执行优先级。因此，显示重排不能改变资源横坐标或同级执行顺序。

4.3 执行系统与节点语义

执行系统每次 Tick 返回 SUCCESS、FAILURE 或 RUNNING。Root 把结果传给唯一孩子；Sequence 从左到右执行，并在失败或运行中的孩子处停止；Selector 在成功或运行中的孩子处停止；Reactive Selector 每次重新检查高优先级分支；Random Selector 在一次运行期间保持已选孩子；Parallel 按配置统计成功与失败；Repeat 控制重复次数或无限重复；Wait 记录经过时间；Action 调用 Actor 方法；Condition 检查 Actor 方法或黑板值。

表 4.1: 当前运行节点的主要语义

节点	主要职责	需要保留的运行记忆
Root	进入整棵树	无
Sequence / Selector	按同级顺序组合结果	当前运行孩子
Reactive Selector	每次从高优先级重新检查	抢占时重置旧分支
Random Selector	随机选择孩子	当前选择和随机状态
Parallel	同时评价多个孩子	各孩子状态
Repeat / Wait	重复或等待	次数或经过时间
Action / Condition	调用 Actor 或读取状态	由具体任务决定

Decorator 在拥有者节点执行前后应用黑板条件、冷却、时间限制、结果取反、强制结果或重复规则。分支被中断时，运行中的节点和 Decorator 都会重置，避免下次进入时继承错误状态。

4.4 编辑工作流与固定基础显示

用户在画布空白处右键创建节点，通过端口建立父子关系，在 Inspector 中使用类型化字段编辑参数。节点可以删除、断开、框选和成组拖动；单击一个节点会清除旧的单选并选中当前节点，框选或带修饰键操作才建立多选。按住空白

处可以平移视窗。所有结构修改、参数修改和用户拖动都进入 Undo/Redo，并可保存为 .tres 后重新加载。

4.5 LIVE DEBUG 与黑板检查

运行游戏时，调试桥为当前 Actor 记录活动节点、节点返回值、失败原因和黑板快照。编辑器把活动路径显示在原行为树画布上，并在侧栏显示黑板键的当前值和类型。失败说明直接标在发生失败的节点附近，例如条件不满足、方法不存在或 Decorator 阻止分支。

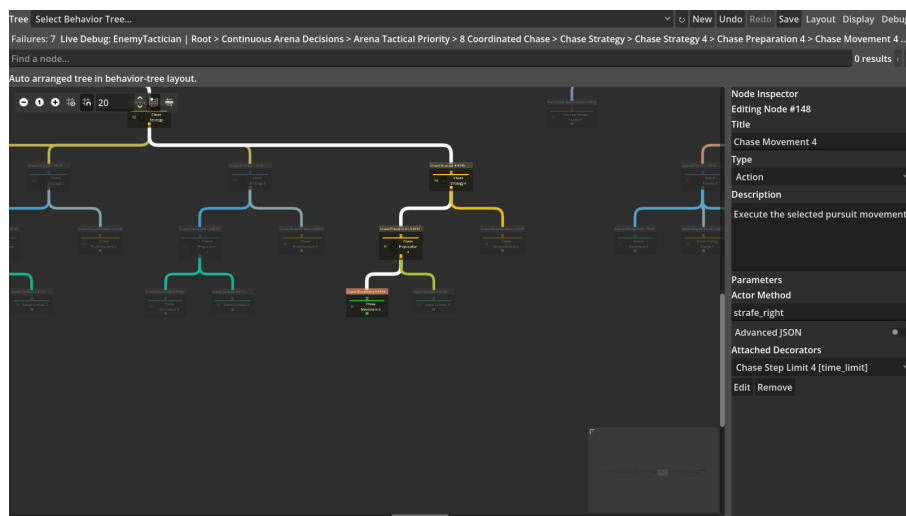


图 4.2: 当前版本在 241 节点真实行为树上显示 Live Debug

4.6 显示优化功能

4.6.1 智能拖拽重排

智能拖拽重排用于解决“一个节点被拖到另一个节点上”造成的新遮挡。系统在累计移动达到 10 个屏幕像素后才启动，并在之后每移动 8 个屏幕像素时更新一次。单击和非常小的修正不会触发重新布局。拖动期间系统直接更新临时避让，所以用户可以在松开鼠标前看到其他卡片让开。

算法先找到与拖动卡片相撞的局部结构，再把连续 2-4 层中的相关父节点、同级节点和孩子作为小组移动。一次最多处理 24 张卡片，并限制碰撞传播次数。这样可以减少整棵树突然排成规则行列的情况。它只维持拖动前已经成立的父节点在上、子节点在下关系；如果用户原本把某组节点故意倒置，系统不会强制纠正。单节点拖动时，智能重排会自动移动附近的卡片，减少拖动造成的遮挡。松开鼠标后，被拖动节点的新位置会保存；其他卡片为了避让而发生的位置变化只影响画面显示，不会修改行为树中保存的坐标。关闭该功能

后，这些卡片会回到原来保存的位置；重建画布时，系统会根据当前保存的坐标重新计算是否需要避让。框选多个节点后，所选节点会作为一个整体移动并保存位置，从而保持它们之间原有的相对关系。

4.6.2 自适应缩放细节

自适应缩放细节把卡片大小和字段数量同时连接到 GraphEdit 缩放。缩放低于 0.62 时使用约 188×88 的紧凑卡片，只保留识别结构所需的低细节；0.62 至 0.88 之间恢复普通尺寸和中等字段；达到 0.88 后显示约 250×150 的普通卡片和完整字段。

卡片尺寸变化后，系统重新检查整张画布的碰撞，并用临时偏移尽量保持原有相对结构。它不是把树永久改成另一套布局，也不是 Smart Drag 的局部求解。放大后字段和普通尺寸会恢复，关闭功能后也会清除自适应尺寸与偏移。

这个功能用信息密度换取概览空间。它的目的不是让缩小后的所有文字仍可阅读，而是让用户先看清树形和分支范围，再在需要时放大查看参数。后面的实验分别记录面积减少、字段减少、完整位于视口内的卡片变化和层级关系。

4.6.3 可读连线覆盖

可读连线覆盖处理连线被非端点卡片挡住的局部情况。开启后，支持的卡片背景把原不透明度乘以 0.72，使卡片后方的直线能够部分显现。文字、图标和控件使用固定前景色，并加 2 px 深色轮廓，避免显现的连线直接穿过字形。关闭时恢复原背景、文字样式和调制颜色。

4.6.4 相关节点聚焦

相关节点聚焦会在用户选择节点后计算结构关系。当前选中的节点使用白色边框；它们的全部祖先和后代使用黄色边框；同一父节点下的其他节点使用绿色边框。其余节点和连线都会明显变淡。

用户需要选择多个节点时，可以在空白处拖出选择框，再一起移动或聚焦。多选聚焦会合并所有选中节点的祖先、后代和同级关系。清除选择后，边框和透明度全部恢复。

4.6.5 鱼眼聚焦

鱼眼聚焦主要用于已经缩小的概览。它会根据节点到指针中心的距离连续改变大小。最靠近中心的节点放大最明显，附近节点也会得到较小的放大；超出主要范围的节点会缩小和淡化。这样做的目的是让用户在不退出概览的情况下看清一个局部区域。

鱼眼只允许距离最近的 8 张卡片参加有限的局部避让，其他节点不会因为焦点变大而推动整张画布。系统使用稳定的参考位置计算距离，并把焦点位置和卡片尺寸分桶，所以鼠标轻微抖动不会每一帧重新排版。滚轮缩放时鱼眼会短暂停止，之后再恢复。

这种设计优先保证焦点和附近节点清楚，而不是保证整棵树完全无重叠。更强的放大可能挡住附近卡片，实验除了记录目标放大，也单独报告新增重叠和层级遮挡。

4.7 测试游戏

测试游戏把五棵不同规模的行为树分配给五名敌人。玩家需要利用移动、跳跃和攻击逐个击败这些敌人。场景包含远程攻击、近战、障碍、跳跃路线、梯子、治疗物品和危险区域。敌人可以巡逻、发现玩家、追击、搜索玩家最后出现的位置、越障、攀爬、撤退和恢复。



图 4.3: 五种规模行为树分别控制可玩场景中的五名敌人

较小的树使用较少分支完成基本行为，较大的树承担更多战斗和环境反应。五名敌人全部被击败后，游戏会进入结束状态。

5 实验结果

5.1 插件功能与可玩游戏验证

自动测试包括执行系统功能、编辑器操作、基础游戏运行、复杂敌人行为和完整游戏流程。所有测试都顺利完成，过程中没有出现报错或程序崩溃。各组的具体数量放在表 5.1 中。

表 5.1: 当前版本的插件功能和游戏验证结果

验证组	通过/总数	主要范围
执行系统功能测试	154/154	节点执行、Decorator、黑板以及行为树的保存和重新加载
编辑器功能测试	337/337	创建和编辑节点、选择和拖动节点，以及显示优化功能的开启和关闭
基础游戏运行测试	40/40	游戏角色能够接收行为树动作，基础场景能够正常运行
复杂游戏行为测试	34/34	敌人的战斗、移动、治疗以及与障碍和梯子的交互
完整游戏流程测试	215/215	五棵不同规模的树能够加载，敌人能够被击败，游戏能够完成
合计	780/780	本次自动检查的全部项目

五棵行为树分别控制对应敌人，游戏能够完成。因此后续显示结果建立在正确加载和执行的真实资源上。

5.2 正式显示数据与安全检查

正式实验形成 225 组功能开启前后的比较。也就是说，同一个显示优化功能会在不同节点数量、不同屏幕尺寸和不同测试目标下反复测试。每一组只改变一个显示优化功能的开关，其余条件保持一致，所以可以比较该功能在当前情况中带来的变化。

所有配对都通过功能与结构检查。行为树的连接关系、执行顺序和保存位置没有被显示优化功能改变。Smart 的测试会在记录后恢复被拖动节点的位置，

所以这里的坐标一致表示自动避让没有写入资源；真实编辑时，用户主动拖动后的新位置仍然会正常保存。

表 5.2: 显示优化功能开启前后的主要结果

显示优化功能	开启前	开启后	主要变化	使用中需要注意的问题
智能拖拽重排	拖动后形成遮挡，平均重叠面积 93,834.90 px ²	45/45 组重叠面积降为 0	消除了测试中主动制造的节点遮挡	平均移动 2.40 张其他卡片，总移动距离中位数 208.54 px
自适应缩放细节	缩小后仍显示完整卡片和字段	总卡片面积平均减少 45.14%	字段平均减少 43.61%，概览更紧凑	低缩放时会主动隐藏部分文字
可读连线覆盖	卡片背景完全不透明，穿过卡片的线容易被挡住	背景不透明度降为 0.72，文字保持不透明	受控遮挡线段中位长度 199.50 px，连线路径不变	只帮助局部连线问题，不减少节点数量
相关节点聚焦	无关节点保持正常亮度	无关节点全部变淡	视口内全亮节点平均减少 58.51%	不合同同时比较多个无关分支
鱼眼聚焦	概览中目标节点平均宽 119.43 px	目标节点平均宽 224.55 px	目标局部明显放大，并恢复更多字段	25/45 组新增局部重叠，28/45 组新增视觉层级遮挡

表 5.2 的每一行看的是不同任务，所以不能把这些数字直接相加成一个总分。下面分别说明每个显示优化功能的效果。所有前后对比图都使用 241 节点行为树、约 27 英寸屏幕配置和同一个测试目标，截图只保留行为树编辑画布，不包含右侧 Inspector。

5.3 显示优化功能的启动前后比较

5.3.1 智能拖拽重排

智能拖拽重排用于处理用户拖动节点时产生的新遮挡。实验中，程序把同一个叶节点拖到另一个节点附近，主动制造遮挡。关闭功能时遮挡会保留下来；开启功能后，周围节点会在不改变行为树资源的情况下避让。

图 5.1 展示了相同拖动位置下的差异。开启后，45 组测试中的重叠面积全部降为 0。它不是把整棵树重新排成固定格式，而是移动与这次拖动有关的一小部分节点。平均每次移动 2.40 张其他卡片，中位数为 1 张，最多为 7 张。所

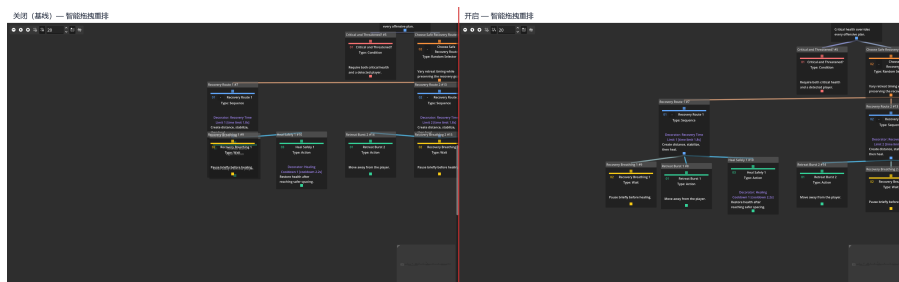


图 5.1: 智能拖拽重排关闭与开启：同一拖动位置下消除卡片遮挡

有其他节点的总移动距离中位数为 208.54 px，最高为 906.37 px。这说明该功能对拖动遮挡很有效，但用户也需要接受少量周围节点会跟着让开。

5.3.2 自适应缩放细节

自适应缩放细节处理的是概览问题。行为树缩小时，如果仍然显示完整卡片，节点会占据大量空间；如果只是把所有东西一起缩小，文字又会很难看清。该功能会在缩小时减少次要字段，并使用更紧凑的卡片。

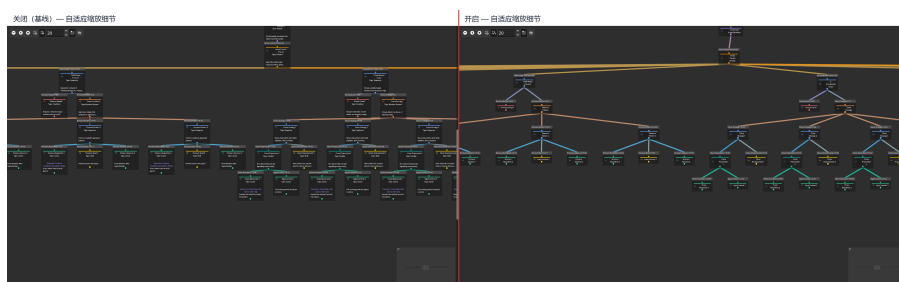


图 5.2: 自适应缩放细节关闭与开启：相同概览缩放下减少卡片面积和字段

图 5.2 中，开启状态保留节点名称、类型和主要结构信息，但不再显示所有参数。实验中，卡片总面积平均减少 45.14%，字段数量平均减少 43.61%，没有新增父节点低于子节点的问题。这个结果说明它适合做大型树的概览基础：先看清树形和分支范围，需要具体参数时再放大查看。

5.3.3 可读连线覆盖

可读连线覆盖解决的是局部连线被卡片挡住的问题。实验分成两部分：第一部分扫描真实视口中有多少可见连线穿过了非端点卡片；第二部分使用同一条真实父子连线和同一张非端点卡片，形成受控遮挡场景，再比较开启前后。

自然视口扫描中，平均每个案例有 3.62 条可见连线被非端点卡片遮挡。受控遮挡中，被挡住的线段中位长度为 199.50 px。开启后，卡片背景变得半透明，但文字和图标仍保持不透明，连线经过的路线也没有改变。因此，它能够让原

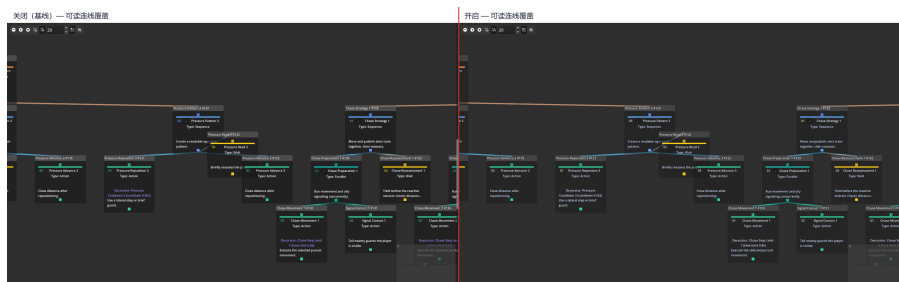


图 5.3: 可读连线覆盖关闭与开启: 相同路线透过卡片背景但避开文字

本被卡片背景盖住的线段露出来一部分。这个功能的作用比较局部, 不能减少节点数量, 也不能让整棵树变得更稀疏。

5.3.4 相关节点聚焦

相关节点聚焦用于查看当前选中节点的结构范围。开启后, 选中节点使用白色边框, 祖先和后代使用黄色边框, 同级节点使用绿色边框, 其他无关节点和连线变淡。因为现在支持框选, 所以实验中也包含一次选择两个节点的情况。

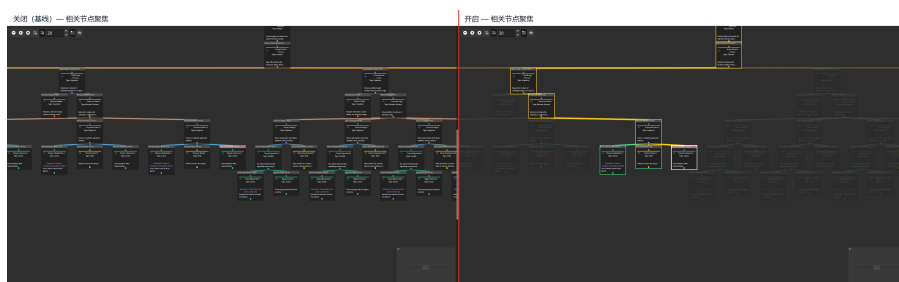


图 5.4: 相关节点聚焦关闭与开启: 选中节点及其关系分支保持突出

结果显示, 所有无关节点都被正确淡化, 视口内仍然保持全亮的候选平均减少 58.51%。节点位置、卡片面积和连线路线没有改变。这样用户仍然知道整棵树大概在哪里, 只是注意力会更容易落在当前选择相关的父节点、子节点和同级节点上。

5.3.5 鱼眼聚焦

鱼眼聚焦主要用于缩小后的概览。关闭状态保留自适应缩放, 开启状态在此基础上放大鼠标附近的节点, 并让远处节点缩小和变淡。因此它测试的是“概览中能不能临时看清某个局部”。

开启后, 目标节点平均宽度从 119.43 px 增加到 224.55 px, 并恢复更多字段。这个变化在视觉上很明显, 尤其是大树被缩得很小时。不过, 鱼眼的代价也最明显: 25/45 组出现新的局部重叠, 28/45 组出现新的视觉层级遮挡。树结构和

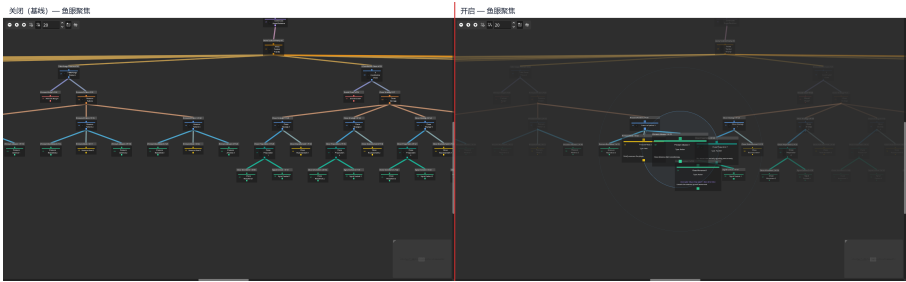


图 5.5: 鱼眼聚焦关闭与开启：概览中恢复目标尺寸并淡化远处节点

执行顺序没有被改变，但画面不再保证全局整洁。因此，鱼眼更适合短时间查看局部，不适合作为一直开启的主布局。

5.4 不同节点规模下的结果

表 5.3 汇总不同节点规模下的结果。为了让表格更容易读，每个显示优化功能只放入最能代表它的指标。Adaptive 看卡片面积减少，Related 看视口中全亮候选减少，Fisheye 看目标宽度增幅，Smart 看自动移动的其他节点数量和总移动距离，Overlay 看真实视口里可见连线被卡片遮挡的比例。

表 5.3: 五种行为树规模下的主要显示变化

资源节点	Adaptive 面积减少	Related 全亮候选减少	Fisheye 宽度增幅	Smart 平均移动节点	Smart 平均总移动距离	Overlay 自然遮挡边比例
31	33.46%	35.41%	37.04%	2.67	441.15 px	0.1052
61	33.58%	57.91%	54.23%	1.67	284.06 px	0.1869
121	46.07%	68.74%	114.94%	1.67	284.06 px	0.1918
241	57.93%	71.38%	226.33%	3.00	435.11 px	0.0218
364	54.67%	59.10%	300.43%	3.00	435.11 px	0.3677

三项规模敏感功能在五棵真实行为树上的平均效果

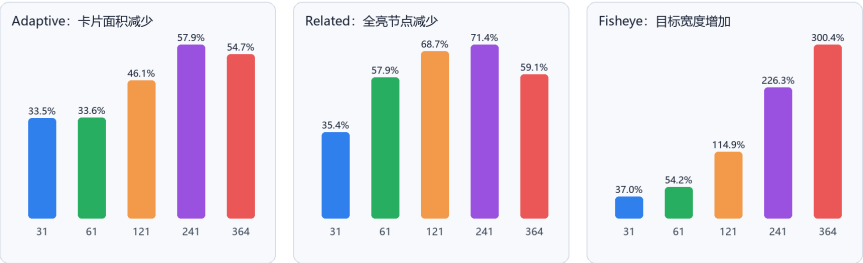


图 5.6: Adaptive、Related Focus 和 Fisheye 随树规模变化的效果

表 5.3 显示，Adaptive 在 121 节点以上的树中更加明显，因为大树在概览中更需要减少卡片面积。Related 在 61 到 241 节点之间逐渐增强，说明树变复杂后，无关分支确实更容易干扰当前选择。364 节点树中的 Related 比 241 节点略低，主要是因为固定目标所在分支不同，画面中相关和无关节点的比例也不同。

单调增加，因为它取决于拖动点附近的局部分支。Overlay 在 364 节点树中自然遮挡比例最高，在 241 节点树中最低，也说明它主要受当前视口和具体连线路径影响，而不是只由节点总数决定。

5.5 不同屏幕尺寸下的结果

表 5.4 汇总三种屏幕尺寸下的结果。这里仍然使用同一批行为树和测试目标，只改变行为树插件视窗的尺寸。

表 5.4: 三种物理尺寸配置下的主要显示变化

屏 幕 配 置	Adaptive 面积 减少	Related 全亮候 选减少	Fisheye 宽度 增幅	Smart 平均 移动节点	Overlay 自然遮 挡边比例
15.94 in	57.14%	54.01%	258.98%	2.40	0.1682
26.96 in	42.61%	59.61%	104.90%	2.40	0.1735
31.55 in	35.67%	61.90%	75.90%	2.40	0.1825

三项屏幕敏感功能在不同物理尺寸配置下的平均效果

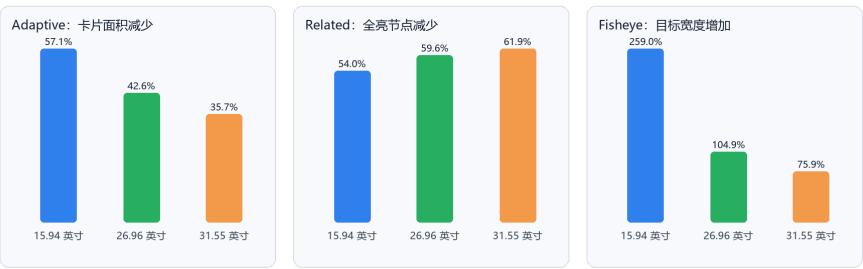


图 5.7: Adaptive、Related Focus 和 Fisheye 随屏幕尺寸配置变化的效果

表 5.4 显示，小屏幕并不是让所有功能都更有效。Adaptive 和 Fisheye 在小屏幕中的相对作用最明显，因为小屏幕中的概览必须把行为树缩得更小。Adaptive 可以先减少全局面积，Fisheye 可以再把当前目标临时放大。因此，它们的作用更像是帮助小屏幕弥补空间不足。

Related 在中屏和大屏上的比例更高。原因是屏幕越大，原本能同时显示的分支越多，其中也包含更多和当前选择无关的内容。Related 不会移动这些节点，而是把无关内容变淡，所以它更适合利用大屏幕的全局空间。Smart 的平均移动节点数量在三种屏幕中相同，说明当前测试里的避让主要由拖动位置决定。Overlay 的自然遮挡比例随屏幕略有上升，因为屏幕越大时能看到的连线更多，也更容易看到某些线穿过卡片。

5.6 结构化开发者评价

表 5.5: 基于统一问题的结构化开发者评价

建议顺序	显示优化功能	最适合的任务和环境	数据支持的优势	观察到的代价	建议
1	自适应缩放细节	小屏及 121-364 节点概览	面积和字段同时下降，无新增层级违反	低缩放隐藏字段	最稳定的通用默认功能
2	智能拖拽重排	各屏幕上的直接拖动编辑	45/45 清除诱发重叠	中位数移动 1 张其他卡片，最多 7 张	编辑时默认开启
3	相关节点聚焦	121 节点以上的结构追踪	淡化全部无关实例	不便同时比较无关分支	选择后默认触发
4	鱼眼聚焦	小屏、241-364 节点的局部查看	目标恢复到约 220px 以上	局部重叠和视觉层级遮挡增加	按需开启
5	可读连线覆盖	特定穿卡片连线	路线不变，背景后方线段可部分显现	作用局部、差异较细	保留为情境功能

表 5.5 没有把不同单位合成一个总分，而是根据实际用途给出建议。自适应缩放细节覆盖范围最大，适合一直作为概览基础。智能拖拽重排在编辑节点位置时最直接。相关节点聚焦适合检查当前节点的结构关系。鱼眼聚焦在小屏和大型树中变化最大，但副作用也更明显。可读连线覆盖适合处理局部连线遮挡，所以它更像补充功能。

6.1 平台正确性是显示研究的前提

表 5.1 中的测试和可通关场景说明，当前树资源、编辑器和执行系统可以一起工作。显示实验也没有改变拓扑、Decorator 归属、同级执行顺序或资源坐标。因此，后面观察到的面积、透明度和目标尺寸变化来自显示优化功能，而不是行为树内容被改动。

不过，这些测试只验证当前节点类型、场景和断言。它们没有证明所有未来 Actor 方法都会正确，也没有把行为树基础功能当成新的研究问题。本文真正讨论的仍然是显示优化功能。

6.2 显示优化作用的解释

6.2.1 不同复杂行为树规模下的优化效果

小型树本来就能保留比较大的卡片，所以这些显示优化功能并不是同样必要。表 5.3 显示，Adaptive、Related 和鱼眼在小型树中的变化都比较有限。Smart 仍然有直接作用，因为一次错误拖动仍会造成遮挡，但小型树通常不需要一直使用鱼眼。

树变大后，卡片密度和无关分支逐渐成为主要问题。Adaptive 先减少整棵树在概览中占用的空间，Related 再让选中的结构从其他分支中突出。两者不是重复功能，而是分别处理“画布太满”和“相关结构不够明显”。这也是它们在中型和大型树中更有价值的原因。

鱼眼在最大两棵树中的局部变化最明显。关闭鱼眼时，目标已经被概览缩得很小；开启后，它会恢复到相近的可读宽度。不过，树越密集，放大的卡片越容易挡住附近内容。因此，鱼眼适合解决“我现在想看清这个局部”，但不适合解决“我希望整棵树一直整齐”。

Smart 和 Overlay 的作用更多取决于局部事件，而不是节点总数。Smart 处理当前拖动造成的碰撞，Overlay 只有在线路穿过卡片时才有明显作用。所以，即使一棵树很大，也不能假设每次拖动或每条连线都会得到相同帮助。

6.2.2 不同物理屏幕尺寸下的优化效果

屏幕结果说明，小屏和大屏需要的帮助不同。小屏概览中的卡片更容易被压得很小，因此 Adaptive 和鱼眼的相对作用最明显。Adaptive 先减少全局占用，鱼眼再恢复当前局部。它们不能把小屏变成大屏，但可以让小屏在查看大型树时少受空间限制。

大屏本来就能同时显示更多内容，所以进一步缩小卡片或放大单个目标的相对变化没有小屏那么大。Related Focus 在这里更有优势，因为它不会丢掉大屏已经显示出的全局位置，只会让无关分支变淡。对大屏来说，这比继续压缩所有节点更符合结构检查任务。

修改后的鱼眼也表现出很直接的权衡：小屏上的局部恢复最明显，新增遮挡也最多。屏幕变大后，焦点原本就更清楚，鱼眼造成的画面变化会更温和。比较合理的用法是在小屏概览中短暂查看目标，完成后退出鱼眼，而不是把它一直当作主布局。

Smart 和 Overlay 在当前实验中没有表现出明显的屏幕差异，因为它们使用的是相同局部操作。这个结果来自同一 GPU 上的尺寸配置重放。真实观看距离、像素密度和输入设备仍然可能改变使用感受。

6.2.3 哪些功能最有价值

从本项目的实际使用来看，如果只能选择一项一直开启的功能，Adaptive 最合适。它在浏览整棵树时持续工作，关闭后也能恢复完整卡片。它的代价很清楚：缩小时看不到全部字段，但用户可以通过放大或鱼眼恢复局部细节。

具体任务的答案并不相同。Smart 在拖动编辑时最直接，因为它处理的是刚刚产生的遮挡。Related 在结构检查时最有价值，因为它不会移动节点，只会把所有选中节点的相关结构保留下来。这两个功能没有必要互相竞争：一个在修改位置时工作，另一个在选择和理解结构时工作。

鱼眼在特定环境中的相对变化最大，但不能因为这一点就排在第一。它的放大效果越强，局部遮挡也越明显，因此更适合作为按需工具。Overlay 的作用最局部，它可以补充被卡片挡住的连线，但不能减少节点、字段或候选数量。

综合来看，Adaptive 适合作为概览基础，Smart 和 Related 分别服务编辑与结构检查，Fisheye 和 Overlay 则在局部问题出现时使用。这个结论比把五个不同单位的百分比相加更接近它们的真实用途。

6.2.4 证据能够说明什么

本研究可以说明这些显示优化功能在当前实现中造成了哪些直接变化，也能指出它们在哪些规模和屏幕下更明显。相同内容的前后图可以帮助读者检查表格与实际画面是否一致，结构签名则确认执行语义没有被改变。

但是，这些数据不能说明普通开发者完成任务快了多少，也不能把卡片面积减少直接写成认知负荷降低。结构化开发者评价只是把当前数据转化为项目设计建议。以后如果能够招募参与者，可以继续使用相同树和任务，再记录完成时间、错误选择、缩放和平移次数，以及主观评分。

7 结论

本项目完成了一个可以编辑、保存、运行和调试的 Godot 4.6 行为树插件，并用它研究大型行为树的显示问题。当前 Display 菜单包含智能拖拽重排、自适应缩放细节、可读连线覆盖、相关节点聚焦和鱼眼聚焦。实验使用五棵真实游戏树和三种来自实际设备的屏幕尺寸配置。

表 5.2 至表 5.4 显示，各项显示优化功能都产生了各自针对的变化，而且没有改变树结构和执行顺序。Smart 适合处理拖动产生的遮挡，Adaptive 适合持续查看整棵树，Related 适合检查选中节点的结构关系。修改后的鱼眼在小屏和大型树上提供了更强的局部放大，但也带来了更多遮挡。Overlay 则只处理特定连线被卡片挡住的情况。

因此，研究问题可以得到有条件的肯定回答：这些功能能够改善它们各自针对的可测显示或编辑状态，但没有一项在所有规模、屏幕和任务中都最好。总体上，Adaptive 最适合作为通用功能，Smart 和 Related 分别适合编辑与结构检查，Fisheye 和 Overlay 更适合按需使用。

本项目的主要价值不是证明行为树优于其他 AI 架构，而是把几种大型图显示方法放进一个真实可运行的行为树编辑器，并允许它们被开启、关闭和比较。后续工作可以让外部游戏开发者完成相同任务，再检查这些几何和显示变化是否真的减少操作时间与错误。

参考文献

Archambault, D. and Purchase, H. C. (2013) ‘The “Map” in the Mental Map: Experimental Results in Dynamic Graph Drawing’, *International Journal of Human-Computer Studies*, 71(11), pp. 1044–1055. Available at: <https://doi.org/10.1016/j.ijhcs.2013.08.004>.

Bederson, B. B., Hollan, J. D., Perlin, K., Meyer, J., Bacon, D. and Furnas, G. W. (1996) ‘Pad++: A Zoomable Graphical Sketchpad for Exploring Alternate Interface Physics’, *Journal of Visual Languages & Computing*, 7(1), pp. 3 – 32. Available at: <https://doi.org/10.1006/jvlc.1996.0002>.

Büring, T., Gerken, J. and Reiterer, H. (2006) ‘User Interaction with Scatterplots on Small Screens: A Comparative Evaluation of Geometric-Semantic Zoom and Fisheye Distortion’, *IEEE Transactions on Visualization and Computer Graphics*, 12(5), pp. 829–836. Available at: <https://doi.org/10.1109/TVCG.2006.187>.

Cockburn, A., Karlson, A. and Bederson, B. B. (2009) ‘A Review of Overview+Detail, Zooming, and Focus+Context Interfaces’, *ACM Computing Surveys*, 41(1), Article 2, pp. 1–31. Available at: <https://doi.org/10.1145/1456650.1456652>.

Colledanchise, M. and Ögren, P. (2017) ‘How Behavior Trees Modularize Hybrid Control Systems and Generalize Sequential Behavior Compositions, the Subsumption Architecture, and Decision Trees’, *IEEE Transactions on Robotics*, 33(2), pp. 372 – 389. Available at: <https://doi.org/10.1109/TRO.2016.2633567>.

Colledanchise, M. and Ögren, P. (2018) *Behavior Trees in Robotics and AI: An Introduction*. Boca Raton, FL: CRC Press. Available at: <https://doi.org/10.1201/9780429489105>.

Di Bartolomeo, S., Crnovrsanin, T., Saffo, D., Puerta, E., Wilson, C. and Dunne, C. (2024) ‘Evaluating Graph Layout Algorithms: A Systematic Review of Methods and Best Practices’, *Computer Graphics Forum*, 43(6), e15073. Available at: <https://doi.org/10.1111/cgf.15073>.

Dwyer, T., Marriott, K. and Stuckey, P. J. (2006) ‘Fast Node Overlap Removal’, in Healy, P. and Nikolov, N. S. (eds.) *Graph Drawing 2005*, LNCS 3843, pp. 153–164. Available at: https://doi.org/10.1007/11618058_15.

Dwyer, T., Marriott, K. and Wybrow, M. (2009) ‘Topology Preserving Constrained Graph Layout’, in Tollis, I. G. and Patrignani, M. (eds.) *Graph Drawing 2008*, LNCS 5417, pp. 230–241. Available at: https://doi.org/10.1007/978-3-642-00219-9_22.

Flórez-Puga, G., Gómez-Martín, M. A., Gómez-Martín, P. P., Díaz-Agudo, B. and González-Calero, P. A. (2009) ‘Query-Enabled Behavior Trees’, *IEEE Transactions on Computational Intelligence and AI in Games*, 1(4), pp. 298 – 308. Available at: <https://doi.org/10.1109/TCIAIG.2009.2036369>.

Furnas, G. W. (1986) ‘Generalized Fisheye Views’, in *Proceedings of CHI ’86*, pp. 16–23. Available at: <https://doi.org/10.1145/22627.22342>.

Gansner, E. R. and Hu, Y. (2010) ‘Efficient, Proximity-Preserving Node Overlap Removal’, *Journal of Graph Algorithms and Applications*, 14(1), pp. 53–74. Available at: <https://doi.org/10.7155/jgaa.00198>.

Ghoniem, M., Fekete, J.-D. and Castagliola, P. (2005) ‘On the Readability of Graphs Using Node-Link and Matrix-Based Representations: A Controlled Experiment and Statistical Analysis’, *Information Visualization*, 4(2), pp. 114 – 135. Available at: <https://doi.org/10.1057/palgrave.ivs.9500092>.

Helmke, S., Doğan, K., Scheffler, R. and Wrobel, G. (2024) ‘Domain-Specific Rules Override Aesthetic Graph Drawing Criteria: An Exploration of User-Generated Diagrams’, in *Diagrammatic Representation and Inference: Diagrams 2024*, LNCS 14981, pp. 44–60. Available at: https://doi.org/10.1007/978-3-031-71291-3_4.

Holten, D. (2006) ‘Hierarchical Edge Bundles: Visualization of Adjacency Relations in Hierarchical Data’, *IEEE Transactions on Visualization and Computer Graphics*, 12(5), pp. 741–748. Available at: <https://doi.org/10.1109/TVCG.2006.147>.

Iovino, M., Scukins, E., Styruud, J., Ögren, P. and Smith, C. (2022) ‘A Survey of Behavior Trees in Robotics and AI’, *Robotics and Autonomous Systems*, 154, 104096. Available at: <https://doi.org/10.1016/j.robot.2022.104096>.

Jakobsen, M. R. and Hornbæk, K. (2013) ‘Interactive Visualizations on Large and Small Displays: The Interrelation of Display Size, Information Space, and Scale’, *IEEE Transactions on Visualization and Computer Graphics*, 19(12), pp. 2336–2345. Available at: <https://doi.org/10.1109/TVCG.2013.170>.

Lim, C.-U., Baumgarten, R. and Colton, S. (2010) ‘Evolving Behaviour Trees for the Commercial Game DEFCON’, in *Applications of Evolutionary Computation: EvoApplications 2010*, LNCS 6024, pp. 100–110. Available at: https://doi.org/10.1007/978-3-642-12239-2_11.

Marzinotto, A., Colledanchise, M., Smith, C. and Ögren, P. (2014) ‘Towards a Unified Behavior Trees Framework for Robot Control’, in *2014 IEEE International Conference on Robotics and Automation*, pp. 5420 – 5427. Available at: <https://doi.org/10.1109/ICRA.2014.6907656>.

McGee, F. and Dingliana, J. (2012) ‘An Empirical Study on the Impact of Edge Bundling on User Comprehension of Graphs’, in *Proceedings of the International Working Conference on Advanced Visual Interfaces*, pp. 620 – 627. Available at:

<https://doi.org/10.1145/2254556.2254670>.

Misue, K., Eades, P., Lai, W. and Sugiyama, K. (1995) ‘Layout Adjustment and the Mental Map’, *Journal of Visual Languages & Computing*, 6(2), pp. 183–210. Available at: <https://doi.org/10.1006/jvlc.1995.1010>.

Plaisant, C., Grosjean, J. and Bederson, B. B. (2002) ‘SpaceTree: Supporting Exploration in Large Node Link Tree, Design Evolution and Empirical Evaluation’, in *IEEE Symposium on Information Visualization*, pp. 57 – 64. Available at: <https://doi.org/10.1109/INFVIS.2002.1173148>.

Purchase, H. C. (1997) ‘Which Aesthetic Has the Greatest Effect on Human Understanding?’ , in Di Battista, G. (ed.) *Graph Drawing, GD 1997*, LNCS 1353, pp. 248–261. Available at: https://doi.org/10.1007/3-540-63938-1_67.

Purchase, H. C., Carrington, D. and Alder, J.-A. (2002) ‘Empirical Evaluation of Aesthetics-Based Graph Layout’, *Empirical Software Engineering*, 7(3), pp. 233–255. Available at: <https://doi.org/10.1023/A:1016344215610>.

Reingold, E. M. and Tilford, J. S. (1981) ‘Tidier Drawings of Trees’, *IEEE Transactions on Software Engineering*, SE-7(2), pp. 223 – 228. Available at: <https://doi.org/10.1109/TSE.1981.234519>.

Sarkar, M. and Brown, M. H. (1994) ‘Graphical Fisheye Views’, *Communications of the ACM*, 37(12), pp. 73–83. Available at: <https://doi.org/10.1145/198366.198384>.

Sugiyama, K., Tagawa, S. and Toda, M. (1981) ‘Methods for Visual Understanding of Hierarchical System Structures’, *IEEE Transactions on Systems, Man, and Cybernetics*, 11(2), pp. 109–125. Available at: <https://doi.org/10.1109/TSMC.1981.4308636>.

Summers, K. L., Goldsmith, T. E., Kubica, S. and Caudell, T. P. (2003) ‘An Experimental Evaluation of Continuous Semantic Zooming in Program Visualization’, in *IEEE Symposium on Information Visualization*, pp. 155 – 162. Available at: <https://doi.org/10.1109/INFVIS.2003.1249021>.

van Ham, F. and Perer, A. (2009) ‘Search, Show Context, Expand on Demand: Supporting Large Graph Exploration with Degree-of-Interest’, *IEEE Transactions on Visualization and Computer Graphics*, 15(6), pp. 953 – 960. Available at: <https://doi.org/10.1109/TVCG.2009.108>.

Ware, C. and Bobrow, R. (2005) ‘Supporting Visual Queries on Medium-Sized Node-Link Diagrams’, *Information Visualization*, 4(1), pp. 49 – 58. Available at: <https://doi.org/10.1057/palgrave.ivs.9500090>.

Ware, C., Purchase, H., Colpoys, L. and McGill, M. (2002) ‘Cognitive Measurements of Graph Aesthetics’, *Information Visualization*, 1(2), pp. 103–110. Available at: <https://doi.org/10.1057/palgrave.ivs.9500013>.

Wang, Y., Wang, Y., Zhang, H., Sun, Y., Fu, C.-W., Sedlmair, M., Chen, B. and Deussen, O. (2019) ‘Structure-Aware Fisheye Views for Efficient Large Graph Explo-

ration’ , *IEEE Transactions on Visualization and Computer Graphics*, 25(1), pp. 566 – 575. Available at: <https://doi.org/10.1109/TVCG.2018.2864911>.

Wong, N., Carpendale, S. and Greenberg, S. (2003) ‘EdgeLens: An Interactive Method for Managing Edge Congestion in Graphs’ , in *IEEE Symposium on Information Visualization*, pp. 51–58. Available at: <https://doi.org/10.1109/INFVIS.2003.1249008>.

Zhou, Z., Jin, J., Phadnis, V., Yuan, X., Jiang, J., Qian, X., Wright, K., Sherwood, M., Mayes, J., Zhou, J., Huang, Y., Xu, Z., Zhang, Y., Lee, J., Olwal, A., Kim, D., Iyengar, R., Li, N. and Du, R. (2025) ‘InstructPipe: Generating Visual Blocks Pipelines with Human Instructions and LLMs’ , in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, Article 877, pp. 1 – 22. Available at: <https://doi.org/10.1145/3706598.3713905>.